

Asof Feature Leakage

Tensium Samples

Pack identification

Category

CPU ML engineering

Pack / category name

Asof Feature Leakage

Why current models fail and the capability this task builds

Every calibration run repaired the pipeline and cleared the forced recovery, yet all four failed on the exact terminal (terminal_value_mismatch). The blocker is point-in-time reasoning under sparse feedback: inferring the full as-of attribution policy, credit notes carried back to the reversed order's date, the settlement embargo, and the tie-breaks, and getting every rule right at once from worked examples and a handful of rate-limited binary checks, instead of probing the oracle. A model that closes this gap gains dependable temporal-leakage correctness: the core skill behind trustworthy feature pipelines and backtests.

1. Description

Description

A hands-on CPU machine-learning engineering task built around point-in-time feature correctness in a repeat-purchase pipeline. The agent investigates misleadingly healthy backtests, infers the true credit-attribution policy from gated evidence, repairs the pipeline, deploys it, recovers from an injected stale-cache incident, validates against a hidden audit slice, and submits a production candidate.

2. Task type & unit type

Task type

Hands-on long-horizon: inspect -> probe -> infer -> edit -> reproduce -> deploy -> validate -> diagnose -> recover -> validate -> promote -> submit.

Unit type

One isolated Harbor task instance containing a real transaction panel, editable sklearn/pandas pipeline, gated operations service, hidden validation slice, frozen terminal and deterministic grader.

3. Verification method

Verification method

Binary reward by behavior. The deployed source snapshot is executed in an unprivileged subprocess over hidden real-data probes. Its canonical integer feature terminal must match a salted frozen oracle. Full reward additionally requires the ordered gate walk, forced stale-cache recovery, minimum interaction depth, clean validation, promotion and submission.

4. Technical domains / adjacent benchmarks

Technical domains

Classical ML engineering; pandas temporal joins and aggregations; point-in-time correctness; returns/credit attribution; feature caching; sklearn backtesting; deterministic offline evaluation.

Adjacent benchmarks

Terminal-Bench-like coding environment with tau-bench-like gated state transitions and hidden deterministic value verification.

5. Human experts used

Human experts

The task was authored and calibrated. Review and QA feedback was supplied by the Tensium task-review team. No additional credential claims are made.

6. Raw base data

Raw base data

UCI Machine Learning Repository Online Retail II transaction data (real invoice lines, 2009–2011). The agent-visible slice contains 594,423 rows and 5,106 customers. The hidden discriminator comes from genuine finance-document timing: credit notes are raised after the commercial event they reverse, creating point-in-time attribution errors during return-heavy periods.

7. What the world looks like

What the world looks like

The agent starts as an unprivileged Ubuntu user in /workspace/target with editable Python sources, real transactions, config, smoke tests and an offline backtest. Network access is disabled. A root-owned Unix-socket broker controls observations, deployment, rate-limited one-bit validation, recovery, promotion and submission. Grader, oracle and hidden probes remain outside the writable workspace.

8. Time per task

Time per task

Approximately 8–10 authoring and QA hours per task.

9. QA process

QA process

QA runs through a systematic, automated pipeline rather than an ad-hoc pass. Deterministic controls mechanically screen the task for the failure modes that matter, reward-hacking and forgeable graders, answer leakage and reachable oracles, in-process grading, and false difficulty (for example, a "failure" that was only budget truncation). Each control is two-condition validated: it must fire on a planted defect and stay silent on a clean task. An adversarial control battery then executes cheat submissions against the real grader and requires every one to score zero, while a gold solution and a structurally different correct solution both score 1.0. An independent review pass grounds every finding in the actual task files, and difficulty is calibrated by running a frontier model at a fixed sampling standard inside the real isolated environment, with guards that separate genuine failures from budget/truncation artifacts.

Completed TASK-QA-CHECKLIST coverage: gold positive control 1.0; seven structurally different negative/cheat controls 0.0; deterministic repeats; grader isolation; no-new-privileges served-path test; package leak guard; secret scan; and real-model trace review. Two behaviorally different model-produced passing candidates demonstrate that grading is by value rather than source shape.

10. Difficulty calibration

SOTA model & pass@1

Claude Opus 5: 4 independent runs; 0/4 strict passes (0%). All four runs scored 0.0 and none completed the gated trajectory. Runs 2 and 3 called finish() after 77 and 56 tool calls; runs 1 and 4 exhausted the agent time budget at 68 and 58 tool calls. No infrastructure failures. The task is currently above the SOTA pass threshold at this tool/time budget.

Rollout records

Final local Claude Code records: asof-feature-leakage_opus5_4traces.zip. Frozen task package (asof-feature-leakage.zip) SHA-256:
7df2e936b6c53087da065a3a2f9c5368b327d2460b9fe57f45a15ce1f0d2ec4f.

11. Grading-design table

Component / gate / axis	What it measures	Weight / cap	How enforced
Env-owned terminal	Exact hidden feature values (n_lines, n_sku, n_credit, spend_cents, days_since_last) from the deployed candidate on unseen (customer_id, as_of) probes	hard requirement	Salted SHA-256 of the canonical terminal (fixed column order, sorted rows, int-cast), recomputed in an isolated setuid(1000) subprocess, compared to the frozen oracle
Ordered gate walk	Required sequence reproduce -> deploy -> validate -> diagnostics.read -> recovery.apply -> validate -> promote -> submit	hard requirement	Subsequence match over the root-owned actions.jsonl ledger (0600), plus correct recovery family and a clean validate recorded after recovery
Forced recovery	Dirty validation observed -> diagnostics.read run -> recovery.apply --family stale_cache	hard requirement	Broker enforces the dependency structurally; validation returns clean only once recovered is set and deployed code is correct
Interaction depth	Genuine long-horizon investigation + operations	hard requirement	≥ 23 accepted actions and ≥ 15 observations, of which the first 15 are exactly inspect, profile, sample_records[0..12], over a gap-free authoritative seq
Isolation / anti-cheat	Candidate cannot read or modify oracle / grader / ledger state	hard $\rightarrow$ 0.0	Unprivileged -l worker (setuid 1000), root-owned oracle+salt+ledger, O_NOFOLLOW snapshot with regular-file/size checks, oracle chmod 0600/root
Final reward	Conjunction of all four requirements	binary 0.0/1.0	terminal_correct AND gate_walk_ok AND forced_recovery_ok AND depth_ok; partial

			diagnostics never compensate
--	--	--	------------------------------

12. How the tasks were created

How created

The task was derived from a real temporal inconsistency in Online Retail II. Author-side analysis measured credit timing and pairing behavior, then built a line-stable bitemporal gold implementation. Thirteen gated worked cases uniquely pin the business rule without disclosing it as prose. Validation and the final terminal use separate hidden probe grids (5,235 and 5,467 rows respectively, 10,702 in total) with separate salted hashes. Validation also corrupts a structurally plausible cached attribution between executions, requiring integrity-aware recovery. A gold trajectory, behaviorally distinct passing implementations, seven negative/cheat controls, Docker isolation tests and four final Claude Opus 5 rollouts were used to validate soundness, fairness and difficulty. The deciding signal is forced through operations; validation results redirect diagnosis and recovery; unrewarded elective depth does not substitute for the required trajectory.

13. QA checklist

- ☒ Two-condition proof: positive control (oracle scores ~ 1.0) AND negative controls (wrong work scores low, for the intended reason).
- ☒ For hands-on tasks: a SECOND, structurally different correct solution also scores ~ 1.0 (grader accepts by behaviour, not gold's shape).
- ☒ Grader isolation: grader, answers, and reference solution live outside the agent's /workspace/target/ (in tests/ and solution/).
- ☒ For hands-on tasks: agent code runs in the verifier as a subprocess with a timeout - never imported into the grader.
- ☒ No answer / solution leakage: nothing in environment/workspace/ reveals the answer; reference deliverable absent; Dockerfile build guard enforces it.
- ☒ Deducibility: everything needed to solve is in the workspace; task is actually solvable; no invisible coin-flips.
- ☒ Real data, not synthetic: built on real OSS repos / public datasets / real experiment artifacts.
- ☒ Long-horizon by design: deciding signal is forced, results redirect the next step, no rewarded elective depth.
- ☒ Build hygiene: zip built in a temp/staging dir; solution/ is excluded from the agent's runtime workspace (present only for reviewer verification).
- ☒ Scrubbed: no secrets/keys, no platform job-ids or URLs, no internal project names or process jargon; no reproduced customer sample content.

- ☑ Realistic and professionally relevant: the task models a production ML pipeline failure caused by point-in-time feature leakage.
- ☑ Clearly specified and internally consistent: the agent instructions, visible tests, gated operations sequence and deterministic grader agree on the required workflow.
- ☑ Reproducible and technically functional: the task uses pinned dependencies, deterministic seeds, frozen hidden probes and containerized verification.
- ☑ Distinct and non-duplicative: the package contains one self-contained task and no duplicate task variants.
- ☑ Licensed and non-sensitive source material: Online Retail II is a public UCI dataset distributed under CC BY 4.0; no private customer material is added.
- ☑ Delivery format: the package follows the Harbor task structure with instruction, task metadata, environment, tests and solution directories.

14. Package overview & contents

This submission covers one task package in detail. The package is self-contained and includes the agent instructions, editable workspace, container environment, hidden verifier assets, deterministic grader and reference solution.

Field	Submission detail
Selected package	asof-feature-leakage.zip
Task count	1 self-contained Harbor task
Task	cpu-ml-engineering/asof-feature-leakage
Task type	CPU ML engineering; hands-on long-horizon investigation, repair, deployment and recovery
Unit / deliverable	One isolated task instance; corrected deployed source snapshot plus successful validation, promotion and submission
Verification	Binary behavioral grading using exact hidden feature values, an ordered gate walk, forced stale-cache recovery and isolated execution
Raw base data	UCI Online Retail II transaction records (CC BY 4.0)

15. Per-task deep-dive

Task	What the task is about	Deliverable
As-of Feature Leakage	Diagnose and repair point-in-time return attribution in a repeat-purchase feature pipeline, then complete the gated deploy, validation, recovery and release sequence.	A corrected deployed pipeline whose canonical hidden feature terminal matches the frozen oracle, followed by

Task	What the task is about	Deliverable
		clean validation, promotion and successful submission.

16. Example prompt

morning need you to pick up something that's been biting us. Our repeat-purchase model's offline backtest looks perfectly healthy, but in production the rankings go sideways during heavy-return stretches, think post-holiday, when the credit notes pile up. Two people already took a swing at it and the number barely moved, so the obvious fix isn't the real one. It smells like a point-in-time / as-of problem where a return is being credited before we'd actually have known about it, but I'll let you confirm that from the data rather than take my word for it.

Everything's in /workspace/target, no internet, deps already installed. The pipeline you can edit is in src/; feel free to read config.yaml, data/transactions.parquet, the data card, scripts/run_backtest.py, and the smoke tests. Poke around first - run inspect, profile, and pull all thirteen sample_records --cursor N examples, and work out the one allocation and point-in-time rule they imply before you touch the code.

Once you've got the fix in, drive it through the release path with python3 env_cli.py: reproduce to deploy to validate to diagnostics.read to recovery.apply --family <the family diagnostics reports> to validate to promote to submit. Heads up: validation is rate-limited and only tells you clean or dirty, and at some point you'll hit a staged cache incident you'll need to diagnose and recover before it'll come back clean. Grading looks at the deployed code, not anything you leave lying around in the workspace - so don't call it done until validation is clean and submit goes through. Thanks!

17. Environment & execution

Requirement	Specification
Runtime	Python 3.12 slim container; unprivileged ubuntu agent workspace
Dependencies	numpy 2.3.5; pandas 2.3.3; scikit-learn 1.7.2; pyarrow 21.0.0; PyYAML 6.0.2; bash, ripgrep, tmux and asciinema
Compute	4 CPUs; 8,192 MB memory; 10,240 MB storage; 0 GPUs
Time limits	Agent: 7,200 seconds; verifier: 1,800 seconds; environment build: 3,600 seconds
Visible tests	python3 -m unittest discover -s tests
Backtest	python3 scripts/run_backtest.py
Operations	python3 env_cli.py OP, followed by the required ordered gate sequence

Requirement	Specification
Verification	tests/test.sh executes python3 /tests/grade.py
Network policy	Internet access disabled

18. Human, AI & automation roles

Contributor	Role in design and production
Human experts	Selected and analyzed the public transaction data; defined the business-rule evidence; authored the environment, oracle and controls; performed security hardening; reviewed failures and approved the frozen task.
AI systems	Used for independent calibration rollouts and behavior testing. Four final Claude Opus 5 runs measured difficulty; behaviorally different passing candidates tested that grading was not tied to one source-code shape.
Automation	Container builds, leak guards, visible smoke tests, isolated hidden probes, canonical salted hashes, action-ledger checks, stale-cache injection, deterministic repeats and negative controls enforce reproducibility and grading integrity.

19. Supporting artifacts

Artifact	Contents
asof-feature-leakage.zip	Complete Harbor task package: instruction, task metadata, environment, workspace, tests, hidden verifier assets and reference solution.
asof-feature-leakage_opus5_4traces.zip	Four independent Claude Opus 5 calibration traces, grader outputs, rewards, action logs, tool counts, statuses and frozen task SHA-256.
Section 13 of this document	Completed quality-control checklist, including positive and negative controls, isolation, leakage, reproducibility, licensing and delivery-format checks.

20. Acknowledgement

The vendor confirms that the task has been reviewed and is realistic, technically substantive, clearly specified and appropriately graded. It reflects a genuine professional workflow and is not a low-effort or artificial task. The creation and calibration statements above are accurate, and the QA checklist has been completed.